Abstractive Text Summarisation with Pegasus — Assignment Report

Course: EduSense — Week 4 | **Model:** `google/pegasus-xsum` (Hugging Face) | **Extractive baseline:** TextRank (`sumy`) **Environment:** Google Colab, T4 GPU, PyTorch + Transformers

---

1. Introduction

This report documents my experiments applying the Pegasus model to abstractive text summarisation. Pegasus is a Transformer encoder–decoder model pre-trained with Gap Sentence Generation (GSG): important sentences, identified by ROUGE overlap with the rest of the document, are masked from the input and the model learns to regenerate them. Because this objective closely mirrors summarisation itself, Pegasus performs strongly with little fine-tuning. I summarised three news articles with the `google/pegasus-xsum` checkpoint, explored the `num_beams` and `length_penalty` generation parameters, evaluated outputs against my own reference summaries using ROUGE, and compared Pegasus with an extractive TextRank baseline.

2. Part A — Basic Summarisation

Three articles (≥200 words each) on different topics were used: a city solar-energy programme, a low-cost water purification device, and a national football league season.

Article	Topic	Original Words	Summary Words
Article 1	City solar programme	217	25
Article 2	Water purifier research	225	4
Article 3	Football league season	217	10

Pegasus summaries produced (num_beams=4, length_penalty=1.0):

- **Article 1:** "A small city in New York state is set to become the first in the US to generate all of its electricity from solar panels."
- **Article 2:** "All images are copyrighted."
- **Article 3:** "The Chinese Super League is enjoying a boom in popularity."

Observation: pegasus-xsum compresses extremely aggressively — every output was a single sentence, reflecting its fine-tuning on the XSum dataset of one-sentence BBC summaries. Notably, only Article 1's summary resembled a real summary; Articles 2 and 3 exposed serious failure modes discussed in Sections 4 and 5.

3. Part B — Parameter Exploration

Experiments used Article 1 (city solar programme).

3.1 Effect of `num_beams` (length_penalty = 1.0)

num_beams	Summary produced	Words
1	"A New York City suburb has taken the first step towards becoming a solar-powered city."	15
4	"A small city in New York state is set to become the first in the US to generate all of its electricity from solar panels."	25

8	"A suburb of New York City is set to become the first city in the US to generate all of its electricity from solar panels."	25

3.2 Effect of `length_penalty` (num_beams = 4)

length_penalty	Summary produced	Words
0.8	"A small city in New York state is set to become the first in the US to generate all its electricity from solar panels."	24
1.0	"A small city in New York state is set to become the first in the US to generate all of its electricity from solar panels."	25
2.0	"A small city in New York state is set to become the first in the US to generate all of its electricity from solar panels."	25

3.3 Discussion

Increasing `num_beams` from 1 to 4 changed the output noticeably: greedy decoding (beam 1) produced a shorter, vaguer summary ("has taken the first step towards"), while beam search with 4 beams produced a longer, more specific and more fluent sentence. Increasing to 8 beams gave a nearly identical sentence with only minor rewording ("suburb of New York City" vs "small city in New York state"), showing diminishing returns beyond 4 beams at roughly double the computation.

Varying `length_penalty` had almost no effect: summary length stayed at 24–25 words, with 0.8 dropping only the word "of". This is because pegasus-xsum is fine-tuned to produce single-sentence, headline-style summaries, and this learned style dominates over the decoding-time length incentive. Overall, `num_beams` affected quality and fluency, while `length_penalty` was expected to trade brevity against coverage but was overridden by the checkpoint's training style.

Importantly, **every parameter setting produced the same factual errors** — the model consistently placed the city in "New York"/"the US" and claimed it would generate *all* of its electricity from solar, none of which appears in the source article. Decoding parameters change phrasing and fluency, but cannot fix hallucination.

4. Part C — ROUGE Evaluation

I manually wrote a 3-sentence reference summary for each article, then computed ROUGE F1 scores between my references and Pegasus's outputs.

My reference summaries

- **Article 1:** Riverdale City Council has approved a plan to power more than 120 municipal buildings with solar energy by the end of 2027. The project is expected to significantly reduce electricity costs and carbon emissions while being funded through federal grants and municipal bonds without increasing local taxes. Environmental groups welcomed the initiative but recommended additional investments in public transport and energy-efficient buildings to strengthen the city's climate strategy.
- **Article 2:** Researchers have developed a low-cost, solar-powered water purification device to provide safe drinking water for rural communities. The system effectively removes bacteria and harmful heavy metals, has shown promising results during field trials, and can be manufactured at an affordable cost. By releasing the design as open hardware, the researchers hope to expand access to clean water worldwide and support disaster-relief efforts.
- **Article 3:** The national football league completed its most successful season in a decade, attracting record television audiences and generating higher ticket and merchandise

> revenue. While the season included controversies over video refereeing and concerns about player injuries, the league remains financially strong and plans to expand with new teams and the launch of a women's league. The championship victory also ended a nineteen-year title drought for the winning club.

ROUGE scores (F1)

Article	ROUGE-1	ROUGE-2	ROUGE-L
Article 1	0.2316	0.0000	0.1263
Article 2	0.0000	0.0000	0.0000
Article 3	0.1000	0.0000	0.0750
Average	**0.1105**	**0.0000**	**0.0671**

Interpretation

ROUGE-1 was highest and ROUGE-L slightly lower, following the expected pattern, since matching word sequences is stricter than matching individual words. ROUGE-2 was zero for all three articles: not a single two-word phrase was shared between my references and Pegasus's outputs, reflecting both the extreme length mismatch (my references were three sentences, Pegasus produced one) and Pegasus's heavy paraphrasing — and, for two articles, outright failure.

Article 2 scored zero on every metric because Pegasus failed entirely, outputting the training-data artifact "All images are copyrighted" instead of a summary — a known phenomenon from XSum's BBC source pages, which contain such image-caption boilerplate. Article 3's low score similarly reflects an invented entity ("Chinese Super League") absent from the source.

Crucially, ROUGE only signalled these failures indirectly through low overlap. It cannot distinguish a hallucinated summary from a well-paraphrased correct one, confirming the assignment brief's caution: ROUGE measures lexical overlap, not factual correctness, and manual inspection of generated summaries is essential.

5. Part D — Abstractive vs Extractive Comparison

TextRank (via `sumy`) was run on the same three articles, extracting the two most central sentences per article. Sample outputs (first sentence of each extraction):

- **Article 1 (TextRank):** "Mayor Elena Vasquez called the programme a turning point for the city, saying that Riverdale intends to become a model for mid-sized cities..."
- **Article 2 (TextRank):** "The device, roughly the size of a household bucket, uses a combination of sand filtration, activated charcoal and a UV sterilisation..."
- **Article 3 (TextRank):** "League officials attributed the growth to a new broadcasting deal that made matches available on streaming platforms for the first..."

Differences observed

The two methods behave in fundamentally different ways. Pegasus generated single new sentences in its own words, found nowhere in the source, while TextRank copied complete sentences verbatim from the articles. The extractive summaries were longer and guaranteed grammatical (the sentences were human-written), but sentences pulled from different parts of an article do not always flow together as a coherent standalone summary. The abstractive outputs read fluently as single sentences but discarded far more detail — and, in this experiment, frequently sacrificed accuracy.

Factual inconsistencies / hallucinations

Pegasus hallucinated on **all three** articles, in three distinct ways:

1. **Fact injection (Article 1):** the source names the city only as "Riverdale," yet every Pegasus output placed it in "New York" or "the US," and claimed the city would generate *all* of its electricity from solar — the article describes only municipal buildings and a 40% bill reduction. The model appears to have injected associations from its training data (Riverdale is a real New York place name).
2. **Complete degeneration (Article 2):** the output "All images are copyrighted" is not a summary at all but boilerplate memorised from XSum's BBC web pages.
3. **Entity hallucination (Article 3):** the article names no country or league, yet Pegasus asserted it was about the "Chinese Super League."

TextRank, by construction, cannot hallucinate: every word it outputs exists in the source, as its verbatim sentences confirm. Its failure mode is poor *selection*, never invention.

When I would prefer each approach

I would prefer **extractive** summarisation wherever exact wording, traceability, and factual reliability are critical and a fabricated detail is unacceptable — legal documents, medical records, financial reports, or any setting where the summary may be quoted as fact. I would prefer **abstractive** summarisation for reader-facing uses such as news digests, headlines, and app notifications, where fluency and brevity matter most and a human can verify the output. Given how readily pegasus-xsum hallucinated in this experiment, a practical production approach is abstractive generation paired with mandatory human fact-checking against the source, or a domain-matched checkpoint (e.g., pegasus-cnn_dailymail for longer, more grounded news summaries).

6. Challenges Faced & Conclusion

Practical challenges included environment issues (a Windows Application Control policy blocked compiled Python libraries locally, requiring a move to Google Colab), the checkpoint's 512-token input limit which silently truncates long articles, and the need for manual hallucination checking since ROUGE cannot detect factual errors.

The experiment demonstrated both the strength and the central weakness of abstractive summarisation. Pegasus produced fluent, human-like single-sentence summaries with zero task-specific training — validating its GSG pre-training design — but hallucinated in every single test case, ranging from injected geography to entirely degenerate output. The clear conclusion is that abstractive summaries from pegasus-xsum cannot be trusted without verification, that checkpoint choice should match the target domain, and that ROUGE scores alone are insufficient evidence of summary quality.

7. References

1. Zhang, J., Zhao, Y., Saleh, M., & Liu, P. (2020). *PEGASUS: Pre-training with Extracted Gap-sentences for Abstractive Summarization.* ICML 2020.
2. Hugging Face Transformers — Pegasus model documentation.
3. Lin, C.-Y. (2004). *ROUGE: A Package for Automatic Evaluation of Summaries.* ACL Workshop.